Natural Scene Image Classification

Rakan Majed Alharbi

1. Introduction

Natural scene image classification is a core problem in computer vision, with applications ranging from autonomous navigation and satellite imagery analysis to content-based image retrieval and photo organization. Unlike object-centric classification tasks, scene classification requires a model to reason about the overall composition of an image — the arrangement, texture, and color distribution of multiple visual elements — rather than a single salient object. This makes it a strong benchmark for evaluating how well a deep learning model can learn hierarchical visual representations.

Convolutional Neural Networks (CNNs) are particularly well suited to this task because their layered structure mirrors the way visual information is naturally organized. Early convolutional layers apply small filters across the image to detect low-level features such as edges, corners, and color gradients. As the network deepens, pooling operations progressively reduce spatial resolution while the convolutional filters combine these low-level responses into increasingly abstract patterns — textures, object parts, and eventually whole scene-level structures such as a mountain ridge, a coastline, or a street grid. This hierarchical feature extraction, learned automatically from data rather than hand-engineered, is what allows a CNN to distinguish between visually similar outdoor scenes such as glaciers and mountains, or between densely built environments such as streets and buildings.

This report documents the design, training, and evaluation of a CNN classifier built to categorize images from the Intel Image Classification dataset into six natural scene classes: buildings, forest, glacier, mountain, sea, and street.

2. Dataset Overview and Class Analysis

The model was trained and evaluated on the Intel Image Classification dataset, publicly available on Kaggle (uploaded by Puneet Bansal, https://www.kaggle.com/datasets/puneet6060/intel-image-classification). The dataset consists of RGB natural scene photographs of size 150×150 pixels, organized into six mutually exclusive classes: buildings, forest, glacier, mountain, sea, and street. The dataset is distributed into three folders: seg_train, seg_test, and seg_pred. In this project, seg_train and seg_test were used for training and evaluation, respectively. The seg_pred directory, used only as unlabeled prediction examples in the original dataset release, contains approximately 7,000 images; since this folder has no ground-truth labels, it cannot contribute to accuracy, precision, recall, or F1-score calculations and was therefore excluded from all evaluation reported in this document.

The seg_train directory contained 14,034 labeled images in total, which were further split 80/20 into a training subset of 11,228 images and a validation subset of 2,806 images (using a fixed random seed for reproducibility). The seg_test directory, used exclusively for final held-out evaluation, contained 3,000 images. The exact per-class breakdown for the training and test folders is shown below.

Class	Train images	Test images
buildings	2,191	437
forest	2,271	474
glacier	2,404	553
mountain	2,512	525
sea	2,274	510
street	2,382	501
Total	14,034	3,000

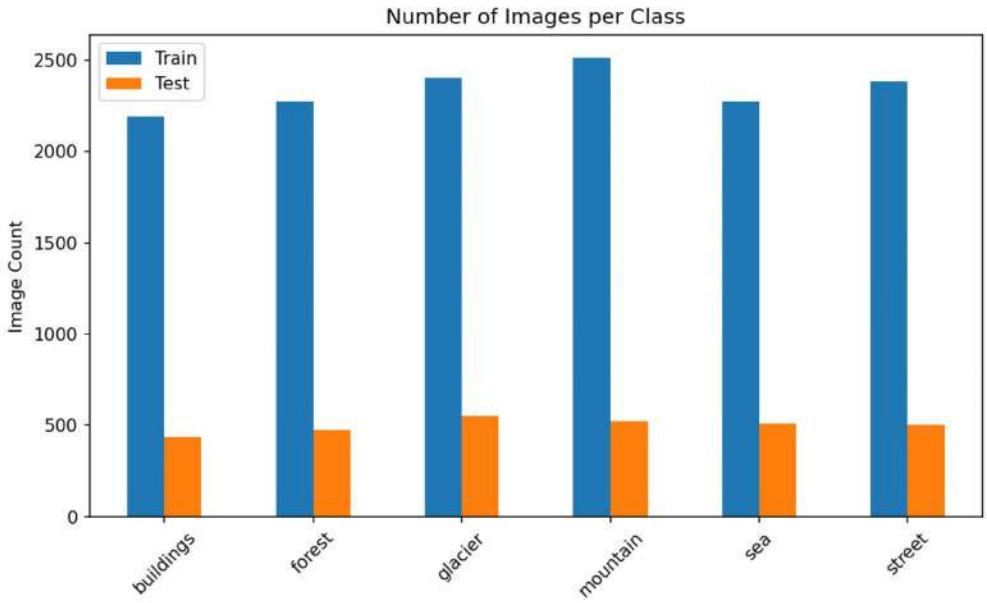


Figure 1. Number of images per class in the training and test sets.

As shown in Figure 1, the class distribution is close to balanced, with training counts ranging from 2,191 (buildings) to 2,512 (mountain) — a spread of about 15%. This mild imbalance is not severe enough to require resampling or class weighting, but it is worth keeping in mind when interpreting per-class performance later in this report.

3. Data Preprocessing

All preprocessing was implemented using TensorFlow's tf.data and Keras utility functions, forming an efficient input pipeline. The following steps were applied:

- Image resizing: every image was loaded and resized to a fixed 150 × 150 resolution using image_dataset_from_directory, matching the native resolution of the dataset and the input size expected by the CNN.
- Color mode: images were kept in RGB (3 channels) rather than converted to grayscale, since color is a discriminative cue for these scene categories — for example, the blue-green tones of sea and glacier images versus the warmer, more varied tones of buildings and street images.
- Label encoding: class labels were inferred automatically from the six subfolder names and encoded as integer labels (label_mode="int"), which were paired with the sparse_categorical_crossentropy loss function rather than one-hot vectors.
- Pixel normalization: raw pixel values in the range [0, 255] were rescaled to [0, 1] using a Rescaling(1/255) layer applied through the pipeline. This keeps gradients well-scaled during training and speeds up convergence.
- Train–validation split: the seg_train folder was split 80% / 20% into training (11,228 images) and validation (2,806 images) subsets using validation_split=0.2 with a fixed seed (42) for reproducibility. The seg_test folder was reserved entirely for final evaluation and was not shuffled, so that predictions could be aligned with true labels for the confusion matrix.
- Batch loading: images were loaded in batches of 32 using tf.data.Dataset objects. The training pipeline additionally applied shuffle(1000) to avoid learning order artifacts, cache() to avoid repeated JPEG decoding after the first epoch, and prefetch(AUTOTUNE) to overlap data loading with model computation.

4. Model Selection

A Convolutional Neural Network (CNN) was selected as the classification architecture, over the other studied alternatives (LeNet, RBM, DBN). CNNs are the natural choice for this task because their convolutional layers preserve the two-dimensional spatial structure of the input image, unlike a Restricted Boltzmann Machine or a Deep Belief Network,

which typically operate on flattened, fully-connected representations and therefore discard local spatial relationships between pixels. Preserving spatial structure is essential for scene classification, where the relative position and arrangement of textures (e.g., sky above a mountain ridge, or a horizon line separating sea from sky) carries meaningful information.

While LeNet is also a convolutional architecture, it was originally designed for small, low-resolution grayscale digits (32 × 32). A deeper CNN with more filters, batch normalization, and dropout — as implemented here — is better suited to the higher-resolution, higher-complexity RGB natural scene images in this dataset, since it can represent a much richer hierarchy of visual features.

5. Model Architecture

The final architecture, named intel_scene_cnn, is a sequential CNN composed of three convolutional blocks followed by a fully-connected classifier head:

- Input layer: accepts 150 × 150 × 3 RGB images.
- Convolutional Block 1: Conv2D(32 filters, 3×3, same padding, ReLU activation) → BatchNormalization → MaxPooling2D(2×2).
- Convolutional Block 2: Conv2D(64 filters, 3×3, same padding, ReLU activation) → BatchNormalization → MaxPooling2D(2×2).
- Convolutional Block 3: Conv2D(128 filters, 3×3, same padding, ReLU activation) → BatchNormalization → MaxPooling2D(2×2).
- Classifier head: Flatten → Dense(128 units, ReLU activation) → Dropout(rate = 0.5) → Dense(6 units, softmax activation).

The number of filters doubles at each convolutional block (32 → 64 → 128), allowing the network to extract progressively more abstract spatial features: edges and color gradients in the first block, textures and shapes in the second, and higher-level scene structures in the third. Batch normalization after each convolution stabilizes and accelerates training by normalizing intermediate activations, while each max-pooling operation halves the spatial resolution, reducing both the parameter count of subsequent layers and the model's sensitivity to small translations in the input. The dropout layer (rate 0.5) placed before the output layer is the model's main defense against overfitting, and the final Dense(6, softmax) layer produces a probability distribution over the six mutually exclusive scene classes.

Model Summary (from Keras)

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 150, 150, 32)	896
batch_normalization	(None, 150, 150, 32)	128
max_pooling2d (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_1 (Conv2D)	(None, 75, 75, 64)	18,496
batch_normalization_1	(None, 75, 75, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_2 (Conv2D)	(None, 37, 37, 128)	73,856
batch_normalization_2	(None, 37, 37, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 18, 18, 128)	0
flatten (Flatten)	(None, 41,472)	0
dense (Dense)	(None, 128)	5,308,544
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 6)	774

Total params: 5,403,462 (20.61 MB) **Trainable params:** 5,403,014 (20.61 MB) **Non-trainable params:** 448 (1.75 KB)

The vast majority of the model's parameters (over 98%) reside in the first Dense layer (5,308,544 of 5,403,462 total), which is a direct consequence of flattening the 18 × 18 × 128 feature map (41,472 values) into a fully-connected layer of 128 units. The 448 non-trainable parameters correspond to the moving mean and variance statistics tracked by the three BatchNormalization layers.

6. Loss Function and Optimizer

Since the class labels were encoded as integers (0–5) rather than one-hot vectors, the model was compiled with the sparse_categorical_crossentropy loss function. This loss is mathematically equivalent to standard categorical cross-entropy for a multi-class, single-label problem, but it avoids the memory overhead of one-hot encoding the labels, which is convenient given the batch pipeline built with image_dataset_from_directory.

The Adam optimizer was used, with a baseline learning rate of 1×10^{-3}. Adam was chosen because it maintains adaptive, per-parameter learning rates derived from running estimates of the first and second moments of the gradients. This makes it converge reliably on image data with a heterogeneous parameter landscape (convolutional filters vs. the large fully-connected layer) without requiring a hand-tuned learning-rate schedule, which would otherwise be needed for plain SGD.

7. Training Strategy and Hyperparameter Tuning

The baseline model was trained for up to 30 epochs with a batch size of 32, using the 11,228-image training split and the 2,806-image validation split described in Section 3. Training used EarlyStopping monitoring validation loss with a patience of 5 epochs and restore_best_weights=True, which both prevents wasted computation once the model stops improving and guarantees that the final saved weights are the best-performing ones observed during training, rather than simply the last epoch's. In practice, training stopped after epoch 15 (of the 30 allotted). Training was conducted locally on a Manjaro Linux workstation, with each epoch taking approximately 10–20 seconds after the first (once the tf.data cache was populated), making the full 15-epoch baseline run complete in well under five minutes. Validation loss reached its minimum of 0.6059 at epoch 10 (validation accuracy 0.8061) and did not improve for five consecutive epochs afterward, triggering early stopping and restoring the epoch-10 weights.

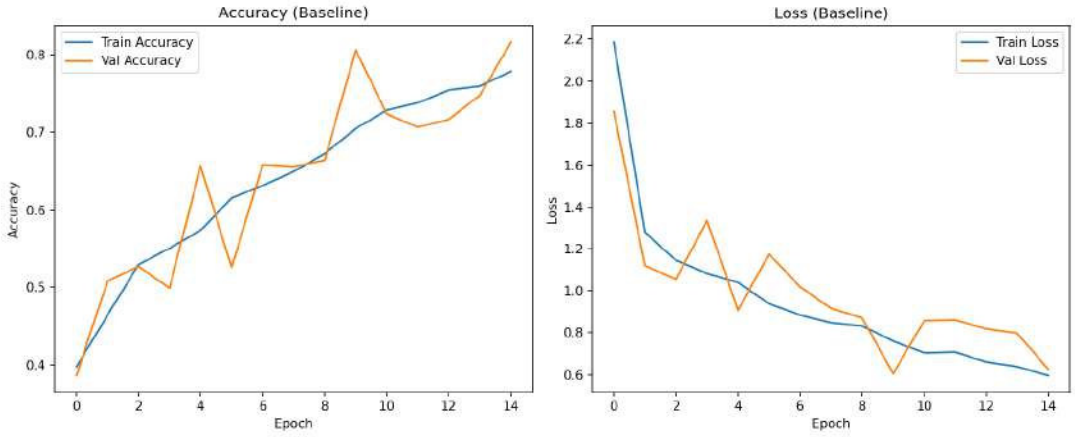


Figure 2. Baseline training and validation accuracy (left) and loss (right) across 15 epochs.

To investigate the effect of key hyperparameters, two additional tuning experiments were run, each training a freshly-initialized model for up to 15 epochs with EarlyStopping patience reduced to 4:

- Experiment 1 — Lower learning rate (LR = 1×10^{-4}, dropout rate unchanged at 0.5): the reduced learning rate produced smoother, more stable convergence, with training accuracy climbing steadily to 0.9093 by epoch 10 and validation accuracy peaking at 0.8186.
- Experiment 2 — Lower dropout (dropout rate = 0.3, learning rate unchanged at 1×10^{-3}): with less regularization, the model reached a best validation accuracy of 0.7958, lower than both the baseline and the lower-learning-rate experiment, and validation loss was noticeably noisier across epochs.

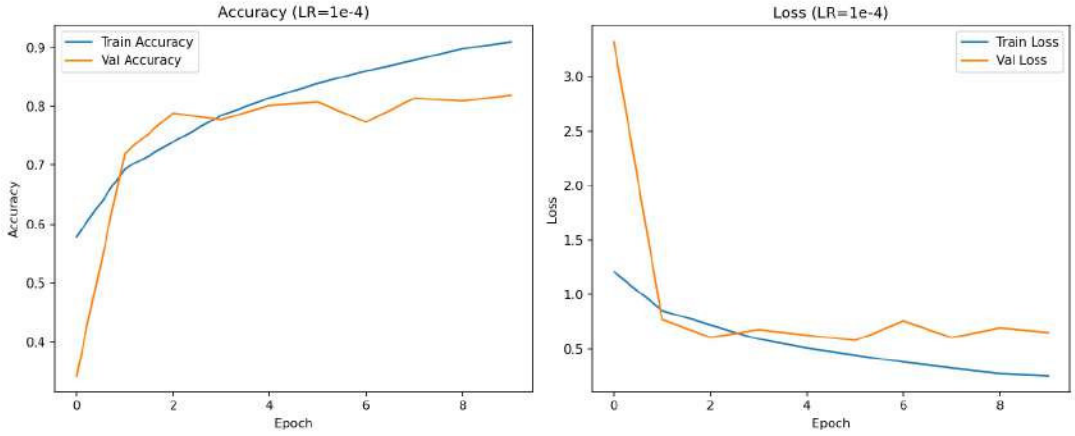


Figure 3. Training and validation accuracy (left) and loss (right) for the LR = 1×10^{-4} tuning experiment.

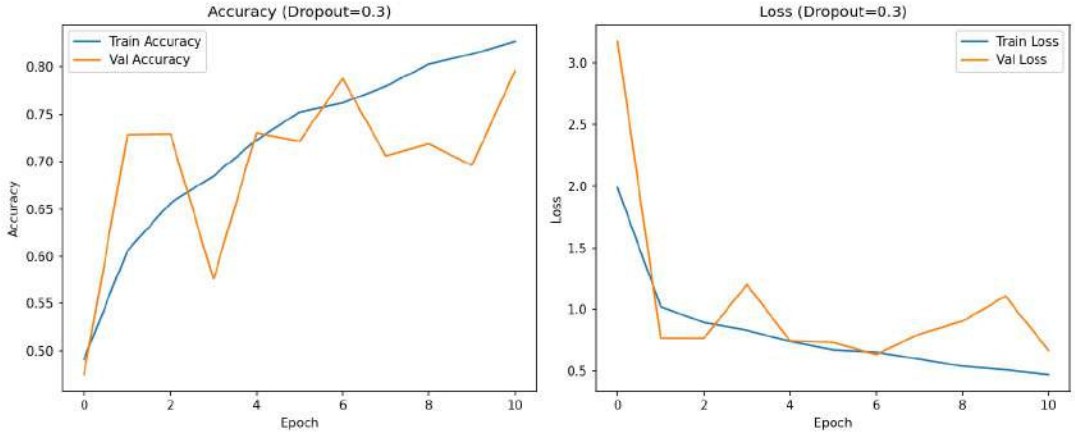


Figure 4. Training and validation accuracy (left) and loss (right) for the Dropout = 0.3 tuning experiment.

Tuning Summary

Experiment	Learning Rate	Dropout Rate	Best Val. Accuracy
Baseline	1×10^{-3}	0.5	0.8158
LR = 1×10^{-4}	1×10^{-4}	0.5	0.8186
Dropout = 0.3	1×10^{-3}	0.3	0.7958

The lower-learning-rate configuration produced the best validation accuracy of the three runs (0.8186), marginally ahead of the baseline (0.8158), while reducing dropout to 0.3 hurt validation performance (0.7958) relative to the baseline's 0.5 dropout rate. This suggests that, for this architecture and dataset size, a moderate-to-high dropout rate is more valuable for generalization than an aggressive learning rate, and that a smaller learning rate trades a slower, more stable convergence for a slightly better validation optimum within the same epoch budget. It is worth noting that the two tuning experiments in this study varied a single hyperparameter at a time (learning rate or dropout rate) relative to the baseline, rather than exploring combinations of both simultaneously. This single-variable approach isolates the individual effect of each parameter clearly, but it cannot capture potential interaction effects—for example, whether a lower learning rate would tolerate a lower dropout rate without the instability observed in Experiment 2, or whether combining the two changes could yield a validation accuracy higher than either change alone. A more exhaustive grid or random search over both parameters jointly is a natural extension of this tuning study and is noted here as a limitation of the current experimental design rather than a limitation of the model architecture itself.

8. Evaluation Metrics and Results

The baseline model (with weights restored to the best validation-loss checkpoint) was evaluated once on the held-out, 3,000-image seg_test set, which was not used during training or validation. The model achieved a test accuracy of 81.30% and a test loss of 0.5749.

Class	Precision	Recall	F1-score	Support
buildings	0.7788	0.8055	0.7919	437
forest	0.9419	0.9578	0.9498	474
glacier	0.8741	0.6528	0.7474	553
mountain	0.6609	0.8762	0.7535	525
sea	0.8221	0.8157	0.8189	510
street	0.8780	0.7904	0.8319	501
Accuracy			0.8130	3,000
Macro avg	0.8260	0.8164	0.8156	3,000
Weighted avg	0.8254	0.8130	0.8132	3,000

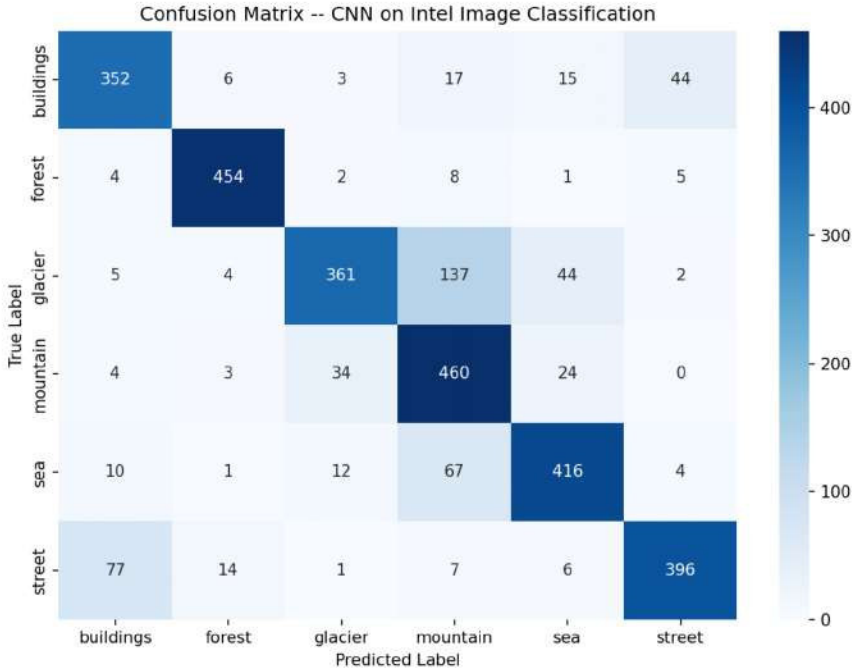


Figure 5. Confusion matrix for the CNN evaluated on the 3,000-image test set.

The forest class was classified almost perfectly (precision 0.9419, recall 0.9578, F1 0.9498), and is visually the most distinctive category in the dataset thanks to its dense green texture, which is rarely confused with the other five classes. Sea and street were also classified reasonably well, with F1-scores of 0.8189 and 0.8319 respectively.

The two weakest classes were glacier (recall 0.6528, F1 0.7474) and mountain (precision 0.6609, F1 0.7535), and they were confused almost exclusively with each other: of the 553 true glacier images, 137 (about 25%) were misclassified as mountain, while of the images predicted as mountain, only 460 out of 696 total mountain predictions were actually mountain images. This is the single largest error pattern in the confusion matrix, and is consistent with the visual overlap between the two categories — both frequently depict rocky, snow-capped terrain against a sky background, differing mainly in the presence and extent of visible ice versus rock, a distinction that can be subtle even for a human observer at 150 × 150 resolution. A secondary but smaller confusion exists between buildings and street (77 street images predicted as buildings, 44 buildings images predicted as street), reflecting the fact that street-level photographs frequently contain building facades in the frame, and vice versa.

9. Regularization and Overfitting Control

Overfitting was controlled primarily through three mechanisms built into the architecture and training loop:

- Dropout (rate = 0.5): applied before the output layer, randomly deactivating half of the 128 dense units on each training step so the network cannot over-rely on specific co-occurring feature combinations.
- Batch normalization: applied after every convolutional layer, which has a mild regularizing effect in addition to its primary role of stabilizing training.
- Early stopping: training was halted once validation loss stopped improving for 5 consecutive epochs (4 for the tuning experiments), with the best-performing weights restored automatically, which prevents the model from continuing to fit noise in the training set after the point of best generalization.

The value of these choices is visible directly in the tuning results from Section 7: reducing dropout from 0.5 to 0.3 (Experiment 2) lowered the best validation accuracy from 0.8158 to 0.7958 and produced a visibly noisier validation loss curve (Figure 4), where validation loss rose and fell sharply from epoch to epoch even as training loss decreased steadily — a classic sign of the training and validation distributions diverging, i.e., overfitting. By contrast, the baseline (0.5 dropout) and the lower-learning-rate experiment showed a tighter, more consistent relationship between training and validation curves (Figures 2 and 3), supporting the choice of 0.5 as the dropout rate for the final model. No data augmentation or L2 weight regularization was used in this project; combining dropout with light data augmentation (random flips or small rotations) is a natural next step, discussed further in Section 10.

10. Challenges and Future Work

The main challenge encountered in this project was the visual similarity between glacier and mountain scenes, which produced the largest single error block in the confusion matrix (137 glacier images misclassified as mountain) and was the primary reason the model's overall macro-averaged F1-score (0.8156) fell short of its overall accuracy (0.8130) — the two classes pull each other's precision and recall in opposite directions (mountain's precision is dragged down by false positives that are actually glacier, while glacier's recall is dragged down by the same errors). A secondary, smaller-scale challenge was the overlap between buildings and street imagery, which share urban visual elements such as walls, windows, and pavement. Training itself was efficient (roughly 10–20 seconds per epoch after the first, thanks to the cached tf.data pipeline) and did not pose a computational challenge; the main practical difficulty was tuning the balance between regularization strength and learning rate to avoid the instability seen in the lower-dropout experiment.

As a future improvement using only the studied basic deep learning methods (i.e., without switching to a pre-trained model), the single most promising next step would be to add data augmentation — random horizontal flips, small rotations, and zoom/crop variations — to the training pipeline. This would expose the model to a wider range of viewpoints and scales for the glacier/mountain and buildings/street pairs without requiring any additional labeled data, and would complement the dropout and batch normalization already in place. A second avenue would be to add a fourth, higher-capacity convolutional block (256 filters) before the dense layer, giving the network more depth to separate glacier and mountain textures at the cost of additional training time and parameters.